Voice-First Drive-Thru Agent

Architecture & Overall Approach (Initial Proposal)

Prepared by: Muhammed Fayiz

1. Objective

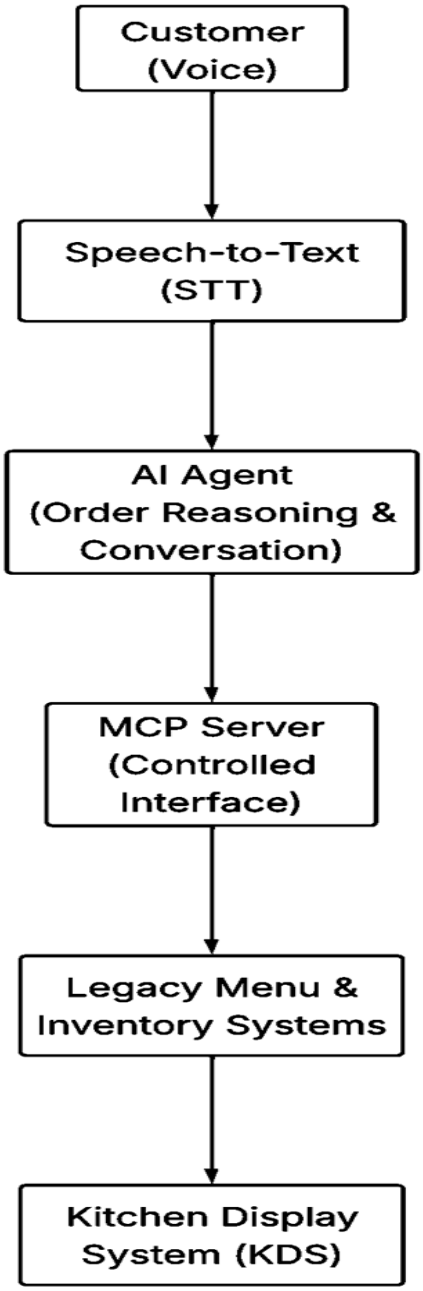
The objective of this project is to design and implement a **Voice-First AI Drive-Thru Agent** for a fast-food chain that can:

- Take customer orders through natural voice interaction
- Handle complex, multi-item orders with modifications
- Check **real-time menu and inventory availability**
- Safely interact with existing **legacy systems**
- Submit confirmed orders to the **Kitchen Display System (KDS)**

2. Key Challenges

1. **Legacy Systems**
    - Menu and inventory data reside on legacy servers not designed for AI access.
    - Direct database or SQL access by AI is unsafe.
2. **Real-Time Inventory**
    - Availability depends on stock, machine status, time rules, and manual overrides.
3. **Voice Ambiguity**
    - Customers change their minds, interrupt, and use informal language.
4. **Operational Safety**
    - Incorrect orders directly impact customers and kitchen operations.

3. High-Level Architecture



4. Component Overview

4.1 Voice Interface Layer

- Captures customer speech
- Converts speech to text (STT)
- Converts AI responses back to speech (TTS)

Responsibility:
Input/output only — no business logic.

4.2 AI Agent (Order Intelligence Layer)

The AI agent is responsible for:

- Understanding customer intent
- Managing conversation flow
- Maintaining order state
- Asking clarifying questions

Important:
The AI agent **does not** access databases or legacy systems directly.

4.3 MCP Server (Model Control Plane)

The MCP Server acts as a **secure, controlled gateway** between the AI agent and legacy systems.

Responsibilities:

- Expose only allowed operations (tools)
- Validate all inputs and outputs
- Enforce business rules
- Normalize legacy responses
- Prevent unsafe actions
- Provide audit logs

Example exposed tools:

- get_menu()
- check_inventory(item_id)
- validate_order(order_payload)
- submit_order(order_payload)

4.4 Legacy Menu & Inventory System

This system represents existing infrastructure that manages:

Menu

- Items
- Categories
- Prices
- Modifiers

Inventory

- Stock levels
- Machine status (e.g., ice cream machine)
- Time-based availability
- Manual overrides by staff

This system is considered **the single source of truth**.

4.5 Kitchen Display System (KDS)

The KDS is the final execution system used by kitchen staff.

Responsibilities:

- Receive finalized orders
- Display items and modifiers clearly
- Track preparation status

The AI **never interacts with the KDS directly**.
All communication is routed through the MCP Server.

5. Order Flow (End-to-End Example)

1. Customer places an order via voice
2. Speech-to-Text converts audio to text
3. AI agent extracts intent and items
4. AI agent queries MCP for:
    - Menu validation
    - Inventory availability
5. MCP queries legacy systems and returns normalized results
6. AI agent confirms the order with the customer
7. AI agent submits the final order to MCP
8. MCP validates and forwards the order to KDS
9. Kitchen staff prepares the order

6. Technology Stack (Proposed)

Backend / MCP Server

- Python
- FastAPI
- Pydantic
- Redis (caching, idempotency)
- PostgreSQL (orders, logs)

AI Agent

- LangGraph (state-based agent orchestration)
- OpenAI / Gemini

Voice

- STT: Whisper / Deepgram
- TTS: ElevenLabs / Azure TTS

7. Development Approach

Phase 1: Text-Only Prototype

- Build agent logic without voice
- Validate order flow and inventory logic

Phase 2: MCP + Mock Legacy Systems

- Simulate menu and inventory responses
- Validate safety and tool contracts

Phase 3: Voice Integration

- Add STT/TTS
- Optimize latency
- Improve user experience

8. Key Design Principles

- AI **never queries databases directly**
- All external access goes through MCP
- Inventory is always checked before confirmation
- Orders are validated twice (before and after confirmation)
- Failures must degrade gracefully